

T2 — АСОИ «Интерактивное обучение электронике и английскому языку»

Автор: Шаболда В.В., гр. АСОИР-221

Конспект для защиты — максимально подробно и «для тупых»: что это вообще такое, как работает, почему сделано именно так.

0. Что это вообще такое, на пальцах

Главная «фишка» T2 — **конструктор логических схем прямо в браузере**: пользователь перетаскивает на экран элементы (вентили И/ИЛИ/НЕ, триггеры, сумматоры, светодиоды...), соединяет их проводами, и видит, как сигналы «бегут» по схеме в реальном времени — загорается лампочка, переключается триггер и т.д. Это аналог настоящего макетного стенда с микросхемами, только виртуальный.

Вокруг этого конструктора — учебная социальная платформа: студенты сдают задания и проверяют работы друг друга (peer review), участвуют в хакатонах, общаются на форуме, учат техническую лексику по английскому (словарь), получают «ачивки» и могут спросить совета у AI-ассистента.

1. Технологии — что это и почему именно так (на пальцах)

1.1 React + SVG для рисования схемы (а не Canvas)

Аналогия: представь два способа рисовать схему.

- **Canvas** — это как рисовать на листе бумаги маркером. Получится красиво, но если пользователь потом кликнет по рисунку, бумага «не знает», что под курсором — программе придётся **самой вычислять**: «курсor в точке (153, 47)... а вентиль AND был нарисован где-то в районе (140-180, 30-60)... наверное, это он?». Эту проверку нужно писать руками для каждой фигуры.
- **SVG** (то, что используется здесь) — это как наклеивать на экран отдельные **стикеры**: каждый вентиль, каждый провод, каждый «порт» (точка подключения) — это отдельный объект (`<rect>`, `<circle>`, `<path>`), который браузер сам отслеживает. Когда пользователь кликает по стикеру — браузер сам говорит программе: «кликнули вот по этому конкретному порту вентиля AND», без всяких вычислений координат руками.

Почему это важно для схемы: у пользователя в схеме нужно кликать по **портам** (чтобы провести провод), по **проводам** (чтобы удалить), по **узлам** (чтобы подвинуть/удалить) — десятки маленьких кликабельных зон. SVG даёт это «из коробки» через обычные React-обработчики `onPointerDown`. А «свечение» активных (включённых) элементов — это просто SVG-фильтр (`filter="url(#glow)"`), как CSS-эффект, без ручной перерисовки.

Canvas был бы выгоднее, если бы на схеме были **тысячи** элементов (производительность важнее удобства кликов) — но в учебных схемах их десятки, поэтому удобство важнее.

1.2 Zustand для состояния схемы (а не Redux / React Context)

Аналогия: представь общую доску, на которой все компоненты экрана «подписаны» на изменения.

- **Redux** — это доска, к которой приколочена строгая инструкция: чтобы изменить ХОТЬ ЧТО-ТО, нужно заполнить бланк («action»), отдать его «диспетчеру» («reducer»), который по правилам обновит доску. Для простого «подвинуть узел на 5 пикселей» — это много лишней бумажной работы.
- **React Context** — это доска, на которую **все** смотрят одновременно: если на доске поменялась хоть одна буква, **все**, кто на неё смотрит, должны заново всё перечитать. А схема при включённом такте (CLOCK) пересчитывается (`runSimulation()`) **каждые 500 мс** — если бы все 50 элементов схемы перерисовывались при каждом тике, это было бы очень тяжело.
- **Zustand** (то, что используется) — это доска, где каждый компонент **подписывается только на свой уголок**: компонент-узел подписан на «свои координаты и значение», компонент-провод — на «своё значение сигнала». Если меняется что-то в другом уголке доски — компонент даже не узнает об этом и не перерисовывается. Минимум кода (`useCircuitStore(s => s.nodes)`), максимум — только нужные перерисовки.

1.3 AI-ассистент на Yandex Foundation Models (YandexGPT-lite) + офлайн-ответы

В T1 для генерации плана обучения используется Gemini (Google). Здесь, в T2, для AI-чата по электронике/английскому используется **другая** модель — YandexGPT-lite (`POST /ai/chat`). Это показывает, что система **не «прибита гвоздями»** к одному поставщику AI — слой общения с AI спрятан за одним эндпоинтом бэкенда, и его можно заменить.

Важная деталь — офлайн-резерв. Прежде чем спрашивать у Yandex (который требует интернет и может быть недоступен), AI-ассистент сначала проверяет **локальный словарь готовых ответов** (`LOCAL_ANSWERS`, ~14 пар «вопрос→ответ») на типовые вопросы: «Что такое XOR?», «Как работает D-триггер?», «Таблица истинности AND», «Как работает сумматор?» и т.п. Если вопрос пользователя похож на один из этих — ответ выдаётся **мгновенно, без сети**. Только если вопрос «нестандартный» — идёт запрос к Yandex API.

1.4 Итеративный алгоритм симуляции (а не «событийная» симуляция или топологическая сортировка)

Это разберём подробно в разделе 3 — здесь только идея «почему так».

Аналогия с альтернативами:

- **Топологическая сортировка** («посчитать всё за один проход слева-направо по порядку зависимостей») — отлично работает, если схема — это «дорога без петель» (направленный ациклический граф). Но в реальных схемах есть **петли**: например, счётчик подаёт сигнал сам на себя через сложение +1. На такой схеме «один проход слева-направо» просто не определён — откуда начинать, если всё друг от друга зависит по кругу?
- **Полноценная событийная симуляция** (как в настоящих SPICE-симуляторах) — моделирует, что сигнал идёт по проводу не мгновенно, а с какой-то задержкой по времени, и обрабатывает

события «сигнал пришёл в момент $t=...$ » по очереди. Это мощно, но сложно реализовать и избыточно для учебного эмулятора.

- **Итеративный подход с «дозаполнением» (fixed-point)** — то, что выбрано здесь: схема пересчитывается **много раз подряд** (до 50 раз), и каждый раз сигналы «продвигаются» на один шаг дальше. Если после очередного прохода **ничего не изменилось** — значит, схема «успокоилась» (достигла стабильного состояния), и можно остановиться. Это просто реализовать, и оно **работает даже с петлями** — просто для петель потребуется больше проходов (или, если петля «бьётся» — НЕ→вход самого себя, — пройдут все 50 итераций, и цикл остановится по лимиту, «зафиксировав» схему в её текущем, нестабильном состоянии).

2. Состав базы данных T2 (что где лежит)

Таблица	Что хранит	Простыми словами
circuit_submissions	Отправленные на проверку схемы + результат	«Журнал посылок» для эмулятора схем
assignments	Задания (включая поле testCases — тесты для автопроверки схем)	Список заданий
assignment_submissions	Сдачи заданий студентами	«Что сдал студент»
peer_reviews	Рецензии студентов на работы друг друга	«Кто кому что написал»
peer_review_sessions	Сессии распределения рецензентов	«Кто кого проверяет в этот раз»
forum_sections, forum_topics, forum_posts	Разделы / темы / сообщения форума	Обычный форум
vocab_terms	Термины технического английского	Карточки словаря
achievements, user_achievements	Достижения и кто их получил	«Ачивки»
hackathons, hackathon_stages, hackathon_tasks, hackathon_teams, ...	Хакатоны, их этапы/задачи/команды	Организация хакатонов

3. Эмулятор цифровых схем — самое главное, разбираем по шагам

3.1 Из чего состоит схема (модель данных)

Представь схему как **список деталей** и **список проводов**:

- **Деталь** (NodeDef) — это, например, «вентиль AND с $id=5$, стоит в координатах (200, 100), имеет 2 входа, текущее значение выхода = false». У некоторых деталей есть дополнительные «личные» поля: у триггера — state (что он запомнил), у счётчика — counter, у мультиплексора — select (какой вход выбран).
- **Провод** (Wire) — это «соединение: выход порта №0 детали 5 идёт на вход порта №1 детали 8».

24 типа деталей разбиты на группы:

- **Логические вентили:** AND, OR, NOT, NAND, NOR, XOR, XNOR — «обычная» комбинационная логика (выход зависит только от текущих входов).
- **Элементы с памятью** (последовательностные): D-триггер (DFF), Т-триггер (TFF), счётчик (COUNTER), регистр сдвига (SHIFT_REGISTER) — их выход зависит не только от текущих входов, но и от того, что «запомнено» внутри.
- **MSI-компоненты** (более сложные «кирпичи», из которых раньше собирали целые блоки): мультиплексор (MUX), дешифратор (DECODER), шифратор (ENCODER), компаратор (COMPARATOR), сумматоры/вычитатели (HALF/FULL ADDER/SUBTRACTOR).
- **Ввод/вывод:** INPUT (переключатель 0/1), OUTPUT, LED (лампочка), CLOCK (генератор тактов), DISPLAY/SEVEN_SEGMENT (цифровые индикаторы).

3.2 Главная функция — `runSimulation()`, два прохода

Каждый «пересчёт» схемы состоит из **двух фаз**.

Фаза 1 — элементы с памятью (по «фронту» такта)

Аналогия — камера с датчиком звонка в дверь. Представь камеру у двери, которая делает фото **только в момент**, когда кто-то нажимает на звонок — то есть в момент, когда сигнал звонка **переходит из «не нажат» в «нажат»** (с 0 на 1). Если звонок просто **удерживают** нажатым — камера не будет фотографировать снова и снова, ей важен только сам **момент перехода**.

Так же работает D-триггер:

- У него два входа: D (что запомнить) и CLK (такт/«звонок»).
- Перед началом пересчёта программа помнит, каким было значение CLK **в конце предыдущего пересчёта** — это prevClk («что было на звонке секунду назад»).
- Если сейчас CLK = 1, а prevClk было 0 — это и есть «момент нажатия звонка» (передний фронт, `clk && !prevClk`). **Только в этот момент** триггер «делает фото»: `state = D` (запоминает текущее значение D).
- Если CLK было и осталось 1 (звонок «удерживают») — `prevClk = 1`, условие `clk && !prevClk = 1 && !1 = 1 && 0 = ложь` — триггер ничего не делает, продолжает хранить то, что запомнил раньше, даже если D сейчас изменился.

Пример по шагам:

Шаг	CLK сейчас	prevCLK	Фронт? (CLK=1 и prevCLK=0)	D	Что делает триггер
1	0	0	нет	1	ничего, state остаётся как было (пусть 0)
2	1	0	да	1	«фото»: state = D = 1
3	1	1	нет	0 (изменился!)	ничего — D изменился, но триггер это «не видит»,

					state остаётся 1
4	0	1	нет	0	ничего, state остаётся 1
5	1	0	да	0	«фото»: state = D = 0

T-триггер работает похоже, но вместо «запомнить D» он **переключается** ($state = !state$) на каждом переднем фронте — как выключатель, который щёлкаешь каждый раз при нажатии кнопки. Счётчик (COUNTER) на каждом фронте увеличивает свой внутренний счётчик на 1 (с учётом сброса RST и разрешения EN).

Фаза 2 — обычная логика (AND/OR/NOT и т.д.), цикл «пока что-то меняется»

Аналогия — слух/сплетня, которая расходится по комнате. Представь комнату с людьми (детали схемы), некоторые держат друг друга за руки (провода). Кто-то в углу «узнал новость» (входной переключатель INPUT включился). На «шаге 1» эту новость узнают только те, кто держит его за руку напрямую. На «шаге 2» новость доходит до тех, кто держит за руку **этих** людей. И так далее — **пока новость не дойдёт до всех, кому положено**, либо пока никто больше не меняет своё «знаю/не знаю».

Алгоритм:

```

изменилось = true; итерация = 0
пока изменилось И итерация < 50:
    изменилось = false; итерация++
    для каждой детали:
        входные_значения = посмотреть, что на входах (по проводам)
        новое_значение = посчитать по таблице истинности (например, AND → вход1 И вход2)
        если новое_значение != старое_значение:
            обновить значение детали; изменилось = true

```

Почему ограничение 50 итераций? Если в схеме есть «дурная петля» — например, вентиль NOT, выход которого проводом соединён со своим же входом — то на каждой итерации значение будет **переключаться**: $0 \rightarrow 1 \rightarrow 0 \rightarrow 1 \rightarrow \dots$ — изменилось будет оставаться true **вечно**. Лимит в 50 итераций — это «выключатель аварийной остановки»: после 50 проходов цикл прерывается насильно, и схема «застывает» в текущем (нестабильном) состоянии, вместо того чтобы программа зависла навечно.

Пример «от начала до конца»: схема $INPUT-A \rightarrow AND \leftarrow INPUT-B$, выход $AND \rightarrow OUTPUT$

Допустим пользователь включил оба переключателя: $INPUT-A = 1$, $INPUT-B = 1$. До запуска симуляции $AND.value = 0$ и $OUTPUT.value = 0$ (значения «по умолчанию»).

Итерация 1 (изменилось сбрасывается в false в начале):

- INPUT-A: у него нет входов, значение не меняется (остаётся 1).
- INPUT-B: аналогично, остаётся 1.
- AND: смотрит на свои входы по проводам → видит [1, 1] (значения INPUT-A и INPUT-B). Считает $AND\ 1 = 1$. Было 0, стало 1 → **изменилось!** $AND.value = 1$, изменилось = true.

- OUTPUT: смотрит на свой вход (провод от AND) → видит 1 (только что обновлённое значение AND, потому что AND обрабатывался раньше по порядку). Было 0, стало 1 → **изменилось!**
OUTPUT.value = 1, изменилось = true.

Итерация 2:

- INPUT-A, INPUT-B: без изменений.
- AND: входы [1, 1], считает 1, уже было 1 → без изменений.
- OUTPUT: вход 1, уже было 1 → без изменений.
- изменилось осталось false → **цикл завершается.**

Итог: за **2 итерации** сигнал «дошёл» от переключателей до лампочки. Если бы схема была длиннее (например, 5 вентилях подряд), потребовалось бы примерно 5 итераций — на каждой итерации сигнал «продвигается» на один вентиль вперёд, как круги по воде.

Финальная синхронизация

После завершения фазы 2 происходит «публикация» внутренней памяти триггеров наружу: DFF.value = DFF.state, TFF.value = TFF.state, COUNTER.value = (counter != 0). То есть «то, что триггер запомнил» (state) и «то, что он показывает на выходном порте» (value) — это разные поля, и наружу (на провода) идёт именно value, синхронизируемое из state в конце каждого пересчёта.

3.3 Многовыходные элементы — сумматоры/вычитатели

У обычного вентиля (AND, OR...) **один** выход — поле value: boolean. Но у сумматора **два** выхода — «сумма» и «перенос» — для них заведён отдельный массив outputValues: boolean[] и константа MULTI_OUTPUT_TYPES = ['HALF_ADDER', 'FULL_ADDER', 'HALF_SUBTRACTOR', 'FULL_SUBTRACTOR'].

Полусумматор (HALF_ADDER) — складывает два бита A и B (как столбиком в школе):

A	B	Сумма (S)	Перенос (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Последняя строка: $1 + 1 = 10$ в двоичной системе — то есть «0 в текущем разряде, 1 переносим в следующий». Формулы:

$$S = A \text{ XOR } B \quad (1, \text{ если } A \text{ и } B \text{ разные})$$

$$C = A \text{ AND } B \quad (1, \text{ только если оба входа — единицы})$$

Полный сумматор (FULL_ADDER) — то же самое, но с дополнительным входом Ci (перенос из предыдущего разряда):

$$S = A \text{ XOR } B \text{ XOR } C_i$$

$$C_o = (A \text{ AND } B) \text{ OR } (C_i \text{ AND } (A \text{ XOR } B))$$

Это позволяет соединять несколько полных сумматоров «цепочкой» (перенос одного — на вход следующего) и собирать многобитный сумматор — классическая демонстрационная задача в курсе электроники.

WireView для таких элементов берёт значение провода не из `node.value`, а из `node.outputValues[slot]` — то есть «какой именно выход» (S или C) идёт по этому конкретному проводу.

3.4 Автоматическая проверка схемы (auto-grading) — как в олимпиадах T1

Идея такая же, как у автоматической проверки решений в олимпиадах (T1): задание содержит набор **тестов** — `Assignment.testCases = [{ inputs: {...}, expectedOutputs: {...} }, ...]`.

Когда студент нажимает «✓ Проверить»:

1. Фронтенд собирает текущую схему (узлы + провода) в JSON и отправляет на POST `/circuit/assignments/:id/submit`.
2. Backend, для **каждого** тест-кейса:
 - берёт копию схемы,
 - подставляет в узлы INPUT значения из `inputs` этого теста,
 - запускает **свою копию** алгоритма из раздела 3.2 (`CircuitService.simulate`),
 - сравнивает значения узлов OUTPUT с `expectedOutputs`.
3. Считается `score/maxScore` (сколько тестов прошло), результат сохраняется в `circuit_submissions` (с `isPassed` и текстовой обратной связью `feedback`).

Почему алгоритм продублирован на сервере? Чтобы студент не мог «обмануть» проверку, подделав результат прямо в браузере (где выполняется тот же алгоритм для визуализации) — итоговую оценку всегда пересчитывает **сервер**, независимо от того, что показал клиент.

Важный честный нюанс: серверная копия алгоритма (`element-registry.ts`) поддерживает **16 типов** элементов — без `HALF_ADDER/FULL_ADDER/DECODER/ENCODER/COMPARATOR/SHIFT_REGISTER/SEVEN_SEGMENT` (они реализованы только в браузерной версии, всего 24 типа). Если в задание добавить тест-кейс со схемой на этих элементах — сервер вернёт ошибку. На практике это не проблема: тест-кейсы создаются под конкретные задания и используют поддерживаемые элементы.

Второй нюанс: сама кнопка «✓ Проверить» в компоненте показывается только если родительская страница передаёт ей функцию `onCircuitSubmit`. На сегодня ни одна страница (`/circuit`, комнаты, страница задания) этого не делает — то есть **backend-цепочка проверки полностью готова и работает**, но кнопка в интерфейсе задания пока не подключена. Это нормально подать как «логика и автопроверка реализованы и

протестированы независимо от UI; подключение кнопки в карточке задания — следующий шаг интеграции».

4. Остальные модули — простыми словами

- **Peer review** (peer_reviews, peer_review_sessions): после того как студент сдал работу, система **распределяет** её на проверку нескольким другим студентам (POST /peer-reviews/assign — по умолчанию reviewsPerStudent = 5). Каждый рецензент видит список «работ на проверку» (GET /peer-reviews/to-review), оценивает по критериям (criteriaScores: jsonb) и оставляет отзыв. Автор работы видит полученные отзывы (GET /peer-reviews/my-received).
- **Хакатоны**: Hackathon → несколько HackathonStage (этапов) → в каждом этапе несколько HackathonTask (задач). Команды (HackathonTeam/HackathonTeamMember) сдают решения (HackathonSubmission/StageSubmission), которые оценивают назначенные проверяющие (TaskReviewer → TaskGrade/HackathonGrade).
- **Форум**: классическая структура раздел → тема → сообщения (forum_sections → forum_topics → forum_posts), с пометками «закреплено» (isPinned), «закрыто» (isClosed), счётчиком просмотров.
- **Словарь технического английского** (vocab_terms): термин, транскрипция, перевод, категория, определение, пример использования, уровень сложности. Фильтрация — по категории и уровню; «поиск» — локальная фильтрация на фронтенде (см. Дб ниже).
- **Достижения** (achievements/user_achievements): 13 типов достижений (первая регистрация, первая сдача, идеальный балл, участие в хакатоне и т.д.) — начисляются автоматически по условиям (conditions: jsonb) и дают очки (points).

5. Разбор графической части (Д1–Д10) — расхождения и как их подать на защите

Д1 — ER-диаграмма (структура БД)

Подтверждено хорошее соответствие для: circuit_submissions, peer_reviews, forum_sections/topics/posts, achievements, user_achievements, vocab_terms, hackathons, hackathon_tasks, hackathon_stages, assignments, assignment_submissions.

Нюансы:

4. ``peer_review_sessions`` не показана на диаграмме, хотя реально существует и используется при распределении рецензентов (POST /peer-reviews/assign). Подача: диаграмма показывает «концептуальное ядро» БД, служебная таблица сессий распределения — деталь реализации.
5. «task_reviewers» (на диаграмме) vs ``@Entity('hackathon_task_reviewers')`` (в коде) — та же сущность, в коде с префиксом hackathon_ для единого нейминга таблиц модуля.
6. **Хакатон-модуль шире показанного**: в коде 10 сущностей (Hackathon, HackathonStage, HackathonTask, HackathonTeam, HackathonTeamMember, HackathonSubmission, HackathonGrade, TaskReviewer, TaskGrade, StageSubmission), на диаграмме, вероятно, показано ядро из 4 — кураторированный срез (как и в Т1).
7. ``assignments.testCases` (jsonb)` — ключевое поле для автопроверки схем (связывает Д1 с Д9). Если его нет на диаграмме — стоит проговорить устно.

8. ``assignment_submissions``: в коде только `createdAt` (без `updatedAt`) — если на диаграмме есть `updated_at`, объясняется тем, что изменения статуса отражаются через явное поле `status`.

Д2 — диаграмма классов

Объединяет поля сущности и методы сервиса в один блок (например, `CircuitSubmission` может показывать `submit()/validateCircuit()`, которые в реализации находятся в `CircuitController/CircuitService`, а не в `Entity`). Стандартное UML-упрощение «сущность + поведение» для наглядности.

Д3 — диаграмма прецедентов

Роли в коде (`UserRoleEnum`): `registered_user`, `student_{english,electronics,computer_science,iot}`, `mentor_{english,electronics,computer_science,iot}`, `admin`. Если на Д3 актёры укрупнены до «Студент/Ментор/Администратор» — нормальное обобщение: в коде роли детализированы по направлениям (общая система для T1/T2/T3), а на уровне прецедентов достаточно укрупнённых ролей с одинаковым набором прав внутри направления.

Д4 — диаграмма компонентов фронтенда

Сверено: ~70 страниц (`frontend/src/app/**/page.tsx`) соответствуют перечню на диаграмме — `VocabularyPage`, `CircuitSimulatorPage` (`app/circuit/page.tsx`), `HackathonDetailPage`, `ForumTopicPage`, `PeerReviewPage`, `AssignmentDetailPage`, `MentorDashboardPage`, `AdminDashboardPage` и т.д. Существенных расхождений не найдено.

Д5 — диаграмма состояний

Реальные статусы: `AssignmentSubmission.status`: `draft` → `submitted` → `under_review` → `reviewed` → `graded`; `CircuitSubmission.isPassed` (бинарный исход); `PeerReviewSession.isDistributed` / `PeerReview.isCompleted`. Если Д5 — про жизненный цикл сдачи задания, путь `draft` → `submitted` → `under_review` → `reviewed` → `graded` и есть его код-основа.

Д6 — последовательность: Словарь

Реально 5 маршрутов (`vocabulary.controller.ts`): `GET /vocabulary` (фильтры `?category=&level=`), `GET /vocabulary/:id`, `POST/PATCH/DELETE`. Отдельного ``/vocabulary/search?q=`` нет — текстового поиска на backend не реализовано. Если Д6 показывает шаг «поиск» как отдельный вызов к `/search` — по факту это локальная фильтрация уже загруженного списка терминов на фронтенде. Подача: «диаграмма отражает пользовательский сценарий поиска; технически поиск — клиентская фильтрация над данными, полученными по категории/уровню».

Д7 — последовательность: Peer Review

Реальные маршруты (`peer-review.controller.ts`): `POST /peer-reviews/assign`, `GET /peer-reviews/to-review`, `GET /peer-reviews/my-received`, `GET /peer-reviews/submission/:submissionId`, `POST /peer-reviews`, `GET /peer-reviews/stats/:assignmentId`. Если на Д7 — `GET /peer-reviews/my-tasks`, `GET /assignments/submissions/:id`, `PATCH /peer-reviews/:id` — это другие названия, но та же логика: `my-tasks` ≈ `to-review`, `assignments/submissions/:id` ≈ `peer-reviews/submission/:submissionId`

(корень ресурса — `peer-reviews`), `PATCH /peer-reviews/:id` $\approx$ `POST /peer-reviews` (создание/сохранение рецензии одним эндпоинтом).

Д8 — последовательность: Материалы (Library)

8 маршрутов (`materials.controller.ts`): `GET /materials` (с `SearchDto`), `GET /materials/public`, `GET /materials/course/:courseId`, `GET /materials/type/:type`, `GET /materials/:id`, `POST /materials`, `PUT /materials/:id`, `DELETE /materials/:id`. Если Д8 показывает `PATCH /materials/:id` — в коде `PUT` (та же операция полного обновления). Если Д8 показывает `GET /materials?courseId=` — в коде это отдельный маршрут `GET /materials/course/:courseId`.

Д9 — последовательность: эмулятор логических схем

Backend-конвейер (`testCases` $\rightarrow$ `POST /circuit/assignments/:id/submit` $\rightarrow$ `validateCircuit` $\rightarrow$ `circuit_submissions`) полностью реализован и работает (см. раздел 3.4). Проверено целиком: `AssignmentDetailPage.tsx` (576 строк) пока содержит только универсальную форму сдачи (текст/репозиторий/файлы), без интеграции эмулятора и кнопки «Проверить» (`onCircuitSubmit` не передаётся). Подача: «самая сложная часть — алгоритм симуляции и автопроверки (включая `MUX/DECODER/ENCODER/COMPARATOR/сумматоры/триггеры`) — реализована и тестируема отдельно; подключение в карточку задания — следующий шаг UI».

Д10 — диаграмма развёртывания

`docker-compose.yml` — 3 сервиса в сети `app-network`: **db** (`postgres:15`, `volume postgres_data`, `healthcheck`), **backend** (NestJS, порт 2904, `healthcheck /health`, монтирует `/var/run/docker.sock` для Docker-песочницы компилятора — общей с T1/T3), **frontend** (Next.js, порт 3000, зависит от backend). Соответствует классической схеме «Клиент $\rightarrow$ Frontend $\rightarrow$ Backend $\rightarrow$ PostgreSQL». Если на Д10 отдельный узел «Docker-sandbox» — это и есть монтирование `docker.sock`.

6. Итоговая «защитная» формулировка (короткая)

T2 — обучающая АСОИ с **интерактивным эмулятором цифровых схем** (React+SVG), в основе которого — двухфазный итеративный алгоритм:

- **Фаза 1:** элементы с памятью (триггеры/счётчики) реагируют только на «передний фронт» такта — сравнение текущего `CLK` с `CLK` из предыдущего пересчёта («камера, фотографирующая только момент нажатия звонка»);
- **Фаза 2:** обычная логика пересчитывается циклически («как слух, расходящийся по комнате»), пока схема не «успокоится» (или до лимита 50 итераций — защита от бесконечных петель).

Эмулятор интегрирован с автопроверкой заданий (`testCases` $\rightarrow$ дублирующий серверный алгоритм $\rightarrow$ `circuit_submissions`), аналогично автопроверке олимпиад в T1 — backend-часть полностью готова, интеграция кнопки в карточку задания — следующий шаг. Вокруг эмулятора — `peer review`, хакатоны, форум, словарь технического английского, достижения и AI-ассистент (YandexGPT-lite с офлайн-резервом из локальной базы ответов).